

DAA Output Of Five Experiments

1. Implementation and Performance Analysis of Interpolation Search

Array: [2, 5, 10, 15, 23, 35, 48, 60, 75, 90, 105, 120]
Searching for: 35
Found at index: 5, Comparisons: 3

Size	IS Time(ms)	BS Time(ms)	IS Comparisons	BS Comparisons
1000	0.0012	0.0009	4	8
5000	0.0034	0.0041	4	12
10000	0.0038	0.0042	5	13
50000	0.0020	0.0018	7	15
100000	0.0031	0.0051	4	16

2. Comparative Analysis of Naive, Rabin-Karp, and KMP Algorithms for String Matching

Text: AABAACAADAABAABA

Pattern: AABA

Naive -> Matches at: [0, 9, 12], Comparisons: 30

KMP -> Matches at: [0, 9, 12], Comparisons: 18

RK -> Matches at: [0, 9, 12], Comparisons: 12

Pattern	Naive	KMP	RK
AB	12492	10000	1302
ABCD	13295	10000	234
ABCDAB	13344	10012	140
ABCDABCD	13341	10012	127

3. Implementation of Kruskal's and Prim's Algorithms for Minimum Spanning Tree

```
=== Kruskal's MST ===
Edge (0 - 3) Weight: 5
Edge (2 - 4) Weight: 5
Edge (3 - 5) Weight: 6
Edge (0 - 1) Weight: 7
Edge (1 - 4) Weight: 7
Edge (4 - 6) Weight: 9
Total MST Cost: 39

=== Prim's MST ===
Edge (0 - 3) Weight: 5
Edge (3 - 5) Weight: 6
Edge (0 - 1) Weight: 7
Edge (1 - 4) Weight: 7
Edge (4 - 2) Weight: 5
Edge (4 - 6) Weight: 9
Total MST Cost: 39
```

4. Implementation of Single Source Shortest Path Algorithm (Dijkstra's)

Shortest paths from vertex 0:

Vertex	Distance	Path
0	0	0
1	3	0 -> 2 -> 1
2	1	0 -> 2
3	4	0 -> 2 -> 1 -> 3
4	7	0 -> 2 -> 1 -> 3 -> 4
5	9	0 -> 2 -> 1 -> 3 -> 4 -> 5

5. To Find Min-Max Value by Applying Divide and Conquer Technique

Array: [3, 1, 7, 4, 9, 2, 8, 5, 6, 0]
Min: 0, Max: 9
D&C Comparisons: 14
Naive Comparisons: 18

Size	DC Comps	Naive Comps	Formula $3n/2-2$
10	14	18	13
100	162	198	148
1000	1510	1998	1498
10000	15902	19998	14998

6. Optimal Cost Computation in Matrix Chain Multiplication using DP Technique

Minimum scalar multiplications: 5000
Optimal parenthesization: ((A1 x A2) x (A3 x A4))

DP Cost Table m[i][j]:

	A	1A	2A	3A	4
A1		0	1500	4500	5000
A2	---		0	9000	4500
A3	---	---		0	3000
A4	---	---	---		0

PS C:\Users\ASUS>

7. Solving N-Queens Problem using Backtracking

```
N=4: 2 solutions, 16 backtracks

All solutions for 4-Queens:

Solution 1: [1, 3, 0, 2]
+---+---+---+---+
| . | Q | . | . |
+---+---+---+---+
| . | . | . | Q |
+---+---+---+---+
| Q | . | . | . |
+---+---+---+---+
| . | . | Q | . |
+---+---+---+---+

Solution 2: [2, 0, 3, 1]
+---+---+---+---+
| . | . | Q | . |
+---+---+---+---+
| Q | . | . | . |
+---+---+---+---+
| . | . | . | Q |
+---+---+---+---+
| . | Q | . | . |
+---+---+---+---+

N=6: 4 solutions, 152 backtracks
N=8: 92 solutions, 2056 backtracks
PS C:\Users\ASUS>
```

8. Travelling Salesman Problem using Branch and Bound for Finding Optimal Path

```
5-City TSP - Cost Matrix:

    A    B    C    D    E
A  INF  10    8    9    7
B   10  INF  10    5    6
C    8   10  INF    8    9
D    9    5    8  INF    6
E    7    6    9    6  INF

Optimal Tour: A -> C -> D -> B -> E -> A
Minimum Cost: 34

Path verification:
A -> C: cost = 8
C -> D: cost = 8
D -> B: cost = 5
B -> E: cost = 6
E -> A: cost = 7
PS C:\Users\ASUS>
```

9. Efficient Bin Packing using Approximation Algorithm

```
Items: [0.5, 0.7, 0.3, 0.9, 0.2, 0.6, 0.8, 0.4, 0.1, 0.5]
Capacity: 1.0
Sum of items: 5.0
Lower bound on bins: 5
```

First Fit (FF): 6 bins

```
Bin 1: [0.5, 0.3, 0.2] | Used: 1.0 [#####]
Bin 2: [0.7, 0.1] | Used: 0.8 [##### ]
Bin 3: [0.9] | Used: 0.9 [##### ]
Bin 4: [0.6, 0.4] | Used: 1.0 [#####]
Bin 5: [0.8] | Used: 0.8 [##### ]
Bin 6: [0.5] | Used: 0.5 [##### ]
```

First Fit Decreasing (FFD): 6 bins

```
Bin 1: [0.9] | Used: 0.9 [##### ]
Bin 2: [0.8, 0.1] | Used: 0.9 [##### ]
Bin 3: [0.7, 0.3] | Used: 1.0 [#####]
Bin 4: [0.6, 0.4] | Used: 1.0 [#####]
Bin 5: [0.5, 0.5] | Used: 1.0 [#####]
Bin 6: [0.2] | Used: 0.2 [#### ]
```

Best Fit Decreasing (BFD): 6 bins

```
Bin 1: [0.9] | Used: 0.9 [##### ]
Bin 2: [0.8, 0.1] | Used: 0.9 [##### ]
Bin 3: [0.7, 0.3] | Used: 1.0 [#####]
Bin 4: [0.6, 0.4] | Used: 1.0 [#####]
Bin 5: [0.5, 0.5] | Used: 1.0 [#####]
Bin 6: [0.2] | Used: 0.2 [#### ]
```

Summary: Lower Bound=5, FF=6, FFD=6, BFD=6

PS C:\Users\ASUS>

10.Improving Quick Sort Efficiency using Randomized Algorithm

Input Type	DQS Comps	DQS Time(ms) ,	RQS Comps	RQS Time(ms)
Random	72980	10.76	67630	9.32
Sorted	12497500	1426.67	75217	13.39
Reverse	12497500	1068.87	70794	8.08
Nearly Sorted	289910	25.32	66399	7.70

PS C:\Users\ASUS>